mixup: BEYOND EMPIRICAL RISK MINIMIZATION

HONGYI ZHANG, MOUSTAPHA CISSE, YANN N. DAUPHIN, DAVID LOPEZ-PAZ

BC. TOMÁŠ HORIČKA, BC. JURAJ BUDINSKÝ

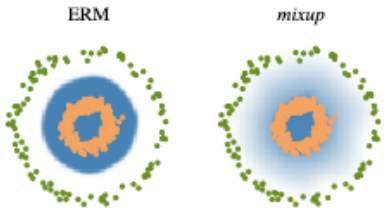
Introduction

- Standard training with ERM learns only from the exact training samples.
- With large neural networks, this can lead to memorization instead of real generalization.
- Models trained this way can also behave unpredictably just outside the training data.
- The paper proposes a very simple fix: train on synthetic examples created between real samples.

Core idea

```
# y1, y2 should be one-hot vectors
for (x1, y1), (x2, y2) in zip(loader1, loader2):
    lam = numpy.random.beta(alpha, alpha)
    x = Variable(lam * x1 + (1. - lam) * x2)
    y = Variable(lam * y1 + (1. - lam) * y2)
    optimizer.zero_grad()
    loss(net(x), y).backward()
    optimizer.step()
```

(a) One epoch of *mixup* training in PyTorch.



(b) Effect of *mixup* ($\alpha = 1$) on a toy problem. Green: Class 0. Orange: Class 1. Blue shading indicates $p(y = 1|x)$.

Figure 1: Illustration of *mixup*, which converges to ERM as $\alpha \to 0$.

- Pick two training examples: (x_i, y_i) and (x_j, y_j)
- Create a new virtual input by interpolation:

$$\tilde{x} = \lambda x_i + (1 - \lambda) x_j$$

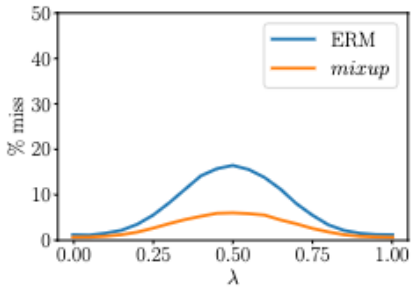
- Create the matching soft target:

$$\tilde{y} = \lambda y_i + (1 - \lambda) y_j$$

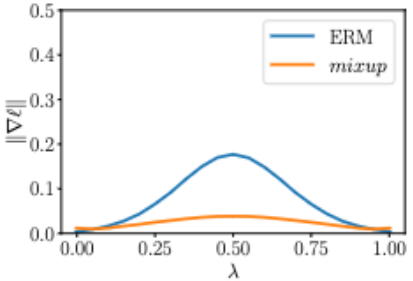
- Here, $\lambda \in [0,1]$ is sampled from a Beta distribution.
- So, the model is trained not only on real points, but also on points between them.

Why it works

- mixup encourages the model to behave more linearly between training examples.
- This acts as a regularizer and reduces sharp, unstable decision boundaries.
- The paper argues that smoother behavior between samples improves generalization.
- On the toy example in Figure 1 and the analysis in Figure 2, mixup produces more stable predictions and smaller gradients between samples.



(a) Prediction errors in-between training data. Evaluated at $x = \lambda x_i + (1-\lambda)x_j$, a prediction is counted as a "miss" if it does not belong to $\{y_i, y_j\}$. The model trained with *mixup* has fewer misses.



(b) Norm of the gradients of the model w.r.t. input in-between training data, evaluated at $x = \lambda x_i + (1-\lambda)x_j$. The model trained with *mixup* has smaller gradient norms.

Figure 2: *mixup* leads to more robust model behaviors in-between the training data.

Why is it practical

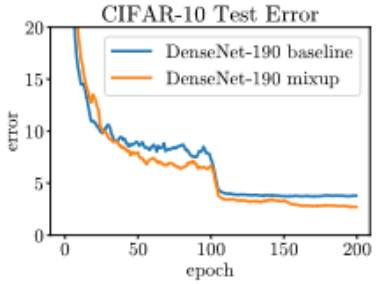
- It is extremely simple to implement.
- It adds only minimal computational overhead.
- It is data-agnostic: it does not require domain-specific augmentation rules.
- In practice, the paper even notes that mixing within a shuffled minibatch works well.

What the paper found

- mixup improved image classification results on CIFAR-10, CIFAR-100, and ImageNet.
- It also improved performance on speech and tabular data.
- It reduced memorization when labels were corrupted.
- It increased robustness against adversarial examples.
- The paper also reports that it can help stabilize GAN training.

Dataset	Model	ERM	*mixup*
CIFAR-10	PreAct ResNet-18	5.6	**4.2**
	WideResNet-28-10	3.8	**2.7**
	DenseNet-BC-190	3.7	**2.7**
CIFAR-100	PreAct ResNet-18	25.6	**21.1**
	WideResNet-28-10	19.4	**17.5**
	DenseNet-BC-190	19.0	**16.8**

(a) Test errors for the CIFAR experiments.



(b) Test error evolution for the best ERM and *mixup* models.

Figure 3: Test errors for ERM and *mixup* on the CIFAR experiments.

Conclusion

- mixup is a simple training principle: learn from interpolations of inputs and labels.
- Its purpose is not just augmentation, but enforcing smoother behavior between samples.
- This leads to better generalization and more robust models.
- The main reason the paper is influential is that such a small change gives benefits across many settings.

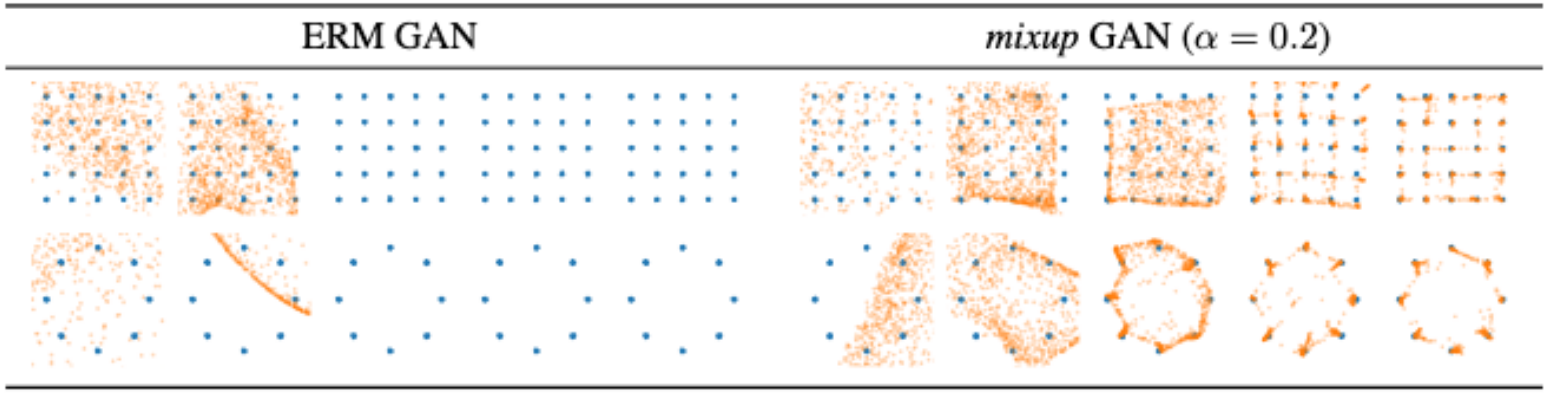


Figure 5: Effect of *mixup* on stabilizing GAN training at iterations 10, 100, 1000, 10000, and 20000.